

Slide 1: Introduction to MongoDB

- **Definition:** MongoDB is a NoSQL, document-oriented database.
- **Why it matters:** It stores data in flexible JSON-like documents instead of tables.
- **Key Terms:** Database, Collection, Document, BSON
- **General Form:**

```
use <database_name>
```

- **Example:**

```
use universityDB
```

Slide 2: SQL vs MongoDB

- **Concept:** Relational vs Document-based storage
- **Purpose:** Understand why MongoDB is flexible
- **General Mapping:**
- Table → Collection
- Row → Document
- Column → Field
- **Example:**

```
{ name: "Ali", age: 20 }
```

- **Tip:** MongoDB allows different fields in documents
-

Slide 3: Databases, Collections, Documents

- **Definition:**
- Database: container
- Collection: group of documents
- Document: actual data
- **General Form:**

```
db.createCollection("students")
```

- **Example:**

```
db.students.insertOne({ name: "Ali" })
```

Slide 4: _id Field

- **Concept:** Unique identifier for every document
- **Why it matters:** Used to find/update/delete data
- **General Form:**

```
{ _id: <value>, field: value }
```

- **Example:**

```
{ _id: 101, name: "Sara" }
```

- **Tip:** Must be unique
-

Phase 1 Summary

- MongoDB stores data as documents
 - Collections do not require fixed schema
 - `_id` uniquely identifies documents
-

Slide 5: Insert Data (Create)

- **Function:** `insertOne()`, `insertMany()`
- **General Form:**

```
db.<collection>.insertOne({ field: value })
```

- **Example:**

```
db.students.insertOne({ name: "Ali", age: 20 })
```

- **Common Mistake:** Wrong data type
-

Slide 6: Read Data (Find)

- **Function:** find(), findOne()
- **General Form:**

```
db.<collection>.find({ condition })
```

- **Example:**

```
db.students.find({ age: { $gt: 18 } })
```

- **Tip:** Use filters carefully
-

Slide 7: Update Data

- **Function:** updateOne(), updateMany()
- **Operators:** \$set, \$inc, \$push
- **General Form:**

```
db.<collection>.updateOne({ filter }, { $set: { field: value } })
```

- **Example:**

```
db.students.updateOne({ name: "Ali" }, { $set: { age: 21 } })
```

Slide 8: Delete Data

- **Function:** deleteOne(), deleteMany()
- **General Form:**

```
db.<collection>.deleteOne({ condition })
```

- **Example:**

```
db.students.deleteOne({ name: "Ali" })
```

- **Tip:** Always double-check condition
-

Phase 2 Summary

- CRUD = Create, Read, Update, Delete
 - Operators control how data changes
-

Slide 9: Data Modeling

- **Concept:** How data is structured
- **Options:** Embedded vs Referenced
- **Example Embedded:**

```
{ name: "Ali", courses: ["DB", "OS"] }
```

- **Tip:** Embed when data is used together
-

Slide 10: Indexing

- **Concept:** Improves query speed
- **General Form:**

```
db.<collection>.createIndex({ field: 1 })
```

- **Example:**

```
db.students.createIndex({ name: 1 })
```

Phase 3 Summary

- Good modeling improves performance
 - Indexing speeds up searches
-

Slide 11: Aggregation Pipeline

- **Concept:** Data processing stages
- **Stages:** \$match, \$group, \$project, \$sort
- **General Form:**

```
db.<collection>.aggregate([ stage1, stage2 ])
```

- **Example:**

```
db.students.aggregate([  
  { $group: { _id: "$department", total: { $sum: 1 } } }  
])
```

Phase 4 Summary

- Aggregation is used for reports
- Similar to SQL GROUP BY

Slide 12: MongoDB with Python

- **Library:** pymongo
- **Example:**

```
collection.insert_one({"name": "Ali"})
```

- **Tip:** Always validate data

Slide 13: MongoDB with Node.js

- **Tools:** MongoDB driver, Mongoose
- **Example:**

```
Student.create({ name: "Ali" })
```

Phase 5 Summary

- MongoDB works with backend apps
- Used in web & desktop apps

Slide 14: Security Basics

- **Concept:** Protect database
 - **Includes:** Users, Roles, Auth
 - **Tip:** Never expose DB publicly
-

Slide 15: Backup & Production

- **Tools:** mongodump, mongorestore
 - **Purpose:** Data safety
-

Phase 6 Summary

- Secure your database
 - Always take backups
-

Final Slide: Conclusion

- MongoDB is flexible and powerful
- Easy for beginners, strong for large systems
- Practice with small projects